AI-Assisted Coding
Lab Assignment - 12.4

Name: A. Sushmitha
Roll No: 2303A51541
Batch: 13

Lab 12 – Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

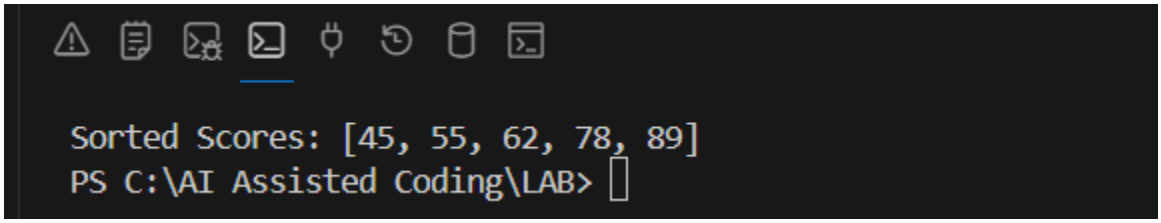
Task 1: Bubble Sort for Ranking Exam Scores
Scenario
You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.
Expected Outcome
• A Bubble Sort implementation with:
- o AI-generated comments explaining the logic
- o Clear explanation of best, average, and worst-case complexity
- o Sample input/output showing sorted scores

Prompt:
Generate a Python program to implement Bubble Sort for sorting student exam scores.
Add inline comments explaining comparisons, swaps, and iteration passes. Include early termination logic and a brief time complexity analysis.

Code:

```
def bubble_sort(scores):
    n = len(scores)
    # Outer loop for number of passes
    for i in range(n):
        swapped = False  # Early termination flag
        # Inner loop for comparisons
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if scores[j] > scores[j + 1]:
                # Swap if they are in wrong order
                scores[j], scores[j + 1] = scores[j + 1], scores[j]
                swapped = True  # Mark that a swap occurred
        # If no swaps happened in this pass, list is already sorted
        if not swapped:
            break
    return scores
# Sample Input
student_scores = [78, 45, 89, 62, 55]
# Sorting the scores
sorted_scores = bubble_sort(student_scores)
# Output
print("Sorted Scores:", sorted_scores)
```

Output:



Time Complexity Analysis:

- ➢ **Best Case:** O(n) – When the list is already sorted, bubble sort only makes one pass through the list without any swaps.
- ➢ **Average Case:** O(n^2) – In most cases, bubble sort compares each element with every other element in the list.
- ➢ **Worst Case:** O(n^2) - When the list is sorted in reverse order, bubble sort must make n passes and perform n comparisons in each pass.

Justification:

1. **Correctness of Algorithm:**
   The program correctly implements Bubble Sort by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. After each pass, the largest unsorted element moves to its correct position at the end of the list.
2. **Early Termination Optimisation:**
   The swapped flag improves efficiency. If no swaps occur during a pass, it means the list is already sorted, so the algorithm stops early. This reduces unnecessary comparisons in the best-case scenario.
3. **Space Complexity:**
   Bubble Sort is an in-place sorting algorithm, so it uses **O(1)** extra space.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario

You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates.

Expected Outcome

• Two sorting implementations:
  o Bubble Sort
  o Insertion Sort

• AI-assisted explanation highlighting efficiency differences for partially sorted datasets

Prompt:
Compare Bubble Sort and Insertion Sort for nearly sorted datasets. Provide Python implementations and explain why Insertion Sort performs better.

Code:

```python
# Bubble Sort Implementation
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False  # Early termination flag
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if arr[j] > arr[j + 1]:
                # Swap if out of order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # Stop early if no swaps happened
        if not swapped:
            break
    return arr

# Insertion Sort Implementation
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]  # Current element to insert
        j = i - 1
        # Shift elements greater than key
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        # Insert key at correct position
        arr[j + 1] = key
    return arr

# Nearly Sorted Attendance Data
attendance = [101, 102, 103, 105, 104, 106, 107]
print("Original Attendance:", attendance)
print("Bubble Sort Result:", bubble_sort(attendance.copy()))
print("Insertion Sort Result:", insertion sort(attendance.copy()))
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-12/task2.py"
Original Attendance: [101, 102, 103, 105, 104, 106, 107]
Bubble Sort Result: [101, 102, 103, 104, 105, 106, 107]
Insertion Sort Result: [101, 102, 103, 104, 105, 106, 107]
PS C:\AI Assisted Coding\LAB>
```

Why Insertion Sort performs better on nearly sorted data:
Insertion sort performs better on nearly sorted data because it minimises the number of comparisons and movements needed to sort the list. When the data is nearly sorted, most elements are already in their correct positions, so Insertion Sort can quickly place each element in its correct position with fewer shifts. In contrast, Bubble Sort will still require multiple passes through the list, even if it is nearly sorted, leading to more comparisons and swaps. Therefore, insertion sort is more efficient for nearly sorted data, often achieving a time complexity close to O(n) in such cases.

Table:

Feature	Bubble Sort	Insertion Sort
Comparisons	More	Fewer
Swaps/Shifts	Multiple passes	Minimal shifts
Nearly Sorted Performance	Moderate	Very Efficient
Practical Speed	Slower	Faster

Justification:
Insertion sort performs better on nearly sorted data because it only shifts misplaced elements into their correct position, requiring fewer operations. Bubble Sort, even with early termination, still makes multiple comparisons across the array. Therefore, for partially sorted datasets like attendance records, insertion sort is more efficient and closer to O(n) performance.

Task 3: Searching Student Records in a Database
Scenario
You are developing a student information portal where users search for student records by roll number.
Expected Outcome
• Two working search implementations with docstrings
• AI-generated explanation of:
    o Time complexity
    o Use cases for Linear vs Binary Search
• A short student observation comparing results on sorted vs unsorted lists

Prompt:
Generate Python implementations of Linear Search and Binary Search for searching student roll numbers. Add docstrings explaining parameters and return values, and compare their time complexity.

Code:

```python
#Implement Linear and Binary Search in Python. Explain when Binary Search is applicable and compare performance on sorted vs unsorted datasets.
# Linear Search Implementation
def linear_search(arr, target):
    """
    Performs a linear search on the given array for the target value.
    Returns the index of the target if found, otherwise returns -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index  # Target found at index
    return -1  # Target not found
# Binary Search Implementation (applicable only on sorted datasets)
def binary_search(arr, target):
    """Performs a binary search on the given sorted array for the target value.
    Returns the index of the target if found, otherwise returns -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2  # Calculate the middle index
        if arr[mid] == target:
            return mid  # Target found at index
        elif arr[mid] < target:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half
    return -1  # Target not found
# Sample Data
sorted_scores = [45, 55, 62, 78, 89]
unsorted_scores = [78, 45, 89, 62, 55]
# Testing Linear Search
print("Linear Search (Sorted):", linear_search(sorted_scores, 62))  # Should return index of 62
print("Linear Search (Unsorted):", linear_search(unsorted_scores, 62))  # Should return index of 62
# Testing Binary Search
print("Binary Search (Sorted):", binary_search(sorted_scores, 62))  # Should return index of 62
print("Binary Search (Unsorted):", binary_search(unsorted_scores, 62))
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-12/task3.py"
Linear Search (Sorted): 2
Linear Search (Unsorted): 3
Binary Search (Sorted): 2
Binary Search (Unsorted): -1
PS C:\AI Assisted Coding\LAB>
```

Table:

Feature	Linear Search	Binary Search
Data Requirement	Works on sorted & unsorted data	Requires sorted data
Best Case	O(1)	O(1)
Average Case	O(n)	O(log n)
Worst Case	O(n)	O(log n)
Suitable For	Small or unsorted lists	Large sorted datasets

Justification:

Binary search is applicable only when the data is sorted because it divides the search space into two halves based on comparison with the middle element. If the dataset is unsorted, binary search cannot correctly determine which half to search, leading to incorrect results. Linear search, however, works on both sorted and unsorted data but takes more time as it checks each element sequentially. For large sorted datasets, binary search is more efficient with O(log n) time complexity, while linear search has O(n) time complexity. Therefore, binary search is preferred when the data is sorted and large in size.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario

You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Expected Outcome

• Fully functional Quick Sort and Merge Sort implementations

• AI-generated comparison covering:

  o Best, average, and worst-case complexities

  o Practical scenarios where one algorithm is preferred over the other

Prompt:

Complete the recursive logic for the given Quick Sort and Merge Sort functions. Add meaningful docstrings and explain how recursion works in both algorithms.

Quick Sort (Partial Implementation)

```
3  def quick_sort(arr):
4      if len(arr) <= 1:
5          return arr
6      else:
7          pivot = arr[0]
8          less_than_pivot = [x for x in arr[1:] if x < pivot]
9          greater_than_pivot = [x for x in arr[1:] if x >= pivot]
10         return quick_sort(less_than_pivot) + [pivot] + quick_sort(greater_than_pivot)
11 # Sample Data
12 scores = [78, 45, 89, 62, 55]
13 # Testing Quick Sort
14 sorted_scores = quick_sort(scores)
15 print("Sorted Scores:", sorted_scores)  # Should return the sorted list of scores
```

Output:

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-12/task4.py"
Sorted Scores: [45, 55, 62, 78, 89]
PS C:\AI Assisted Coding>
```

Code:

```python
def quick_sort(arr):
    """
    Sort an array using the Quick Sort algorithm (recursive).
    Quick Sort works by:
    1. Selecting a pivot element from the array
    2. Partitioning the array into elements less than and greater than the pivot
    3. Recursively sorting each partition
    4. Combining sorted partitions with the pivot
    Time Complexity: O(n log n) average, O(n²) worst case
    Space Complexity: O(n) due to new lists created
    Args:
        arr (list): Unsorted list of comparable elements
    Returns:
        list: Sorted list in ascending order
    Example:
        >>> quick_sort([78, 45, 89, 62, 55])
        [45, 55, 62, 78, 89]
    """
    # Base case: arrays with 0 or 1 element are already sorted
    if len(arr) <= 1:
        return arr
    # Recursive case: partition and sort
    pivot = arr[0]
    less_than_pivot = [x for x in arr[1:] if x < pivot]
    greater_than_pivot = [x for x in arr[1:] if x >= pivot]
    # Recursively sort partitions and combine
    return quick_sort(less_than_pivot) + [pivot] + quick_sort(greater_than_pivot)
# Test 1: Random data
print("=" * 50)
print("TEST 1: Random Data")
print("=" * 50)
random_data = [78, 45, 89, 62, 55, 23, 91, 10]
sorted_random = quick_sort(random_data)
print(f"Original:  {random_data}")
print(f"Sorted:    {sorted_random}")
print(f"Expected:  {sorted(random_data)}")
print(f"✓ PASS" if sorted_random == sorted(random_data) else "✗ FAIL")
```

```python
    # Test 2: Already sorted data
    print("\n" + "=" * 50)
    print("TEST 2: Already Sorted Data")
    print("=" * 50)
    sorted_data = [10, 23, 45, 55, 62, 78, 89, 91]
    sorted_result = quick_sort(sorted_data)
    print(f"Original:  {sorted_data}")
    print(f"Sorted:    {sorted_result}")
    print(f"Expected:  {sorted(sorted_data)}")
    print(f"✓ PASS" if sorted_result == sorted(sorted_data) else "✗ FAIL")
    # Test 3: Reverse-sorted data
    print("\n" + "=" * 50)
    print("TEST 3: Reverse-Sorted Data")
    print("=" * 50)
    reverse_data = [91, 89, 78, 62, 55, 45, 23, 10]
    sorted_reverse = quick_sort(reverse_data)
    print(f"Original:  {reverse_data}")
    print(f"Sorted:    {sorted_reverse}")
    print(f"Expected:  {sorted(reverse_data)}")
    print(f"✓ PASS" if sorted_reverse == sorted(reverse_data) else "✗ FAIL")
    # Test 4: Edge cases
    print("\n" + "=" * 50)
    print("TEST 4: Edge Cases")
    print("=" * 50)
    print(f"Empty list:    {quick_sort([])}")
    print(f"Single element: {quick_sort([42])}")
    print(f"Duplicates:    {quick_sort([5, 2, 5, 1, 2])}")
    # Test 5: Original sample data
    print("\n" + "=" * 50)
    print("TEST 5: Original Sample Data")
    print("=" * 50)
    scores = [78, 45, 89, 62, 55]
    sorted_scores = quick_sort(scores)
    print("Sorted Scores:", sorted_scores)
```

Output:

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-12/task4.py"
==================================================
TEST 1: Random Data
==================================================
Original:  [78, 45, 89, 62, 55, 23, 91, 10]
Sorted:    [10, 23, 45, 55, 62, 78, 89, 91]
Expected:  [10, 23, 45, 55, 62, 78, 89, 91]
✓ PASS

==================================================
TEST 2: Already Sorted Data
==================================================
Original:  [10, 23, 45, 55, 62, 78, 89, 91]
Sorted:    [10, 23, 45, 55, 62, 78, 89, 91]
Expected:  [10, 23, 45, 55, 62, 78, 89, 91]
✓ PASS

==================================================
TEST 3: Reverse-Sorted Data
==================================================
Original:  [91, 89, 78, 62, 55, 45, 23, 10]
Sorted:    [10, 23, 45, 55, 62, 78, 89, 91]
Expected:  [10, 23, 45, 55, 62, 78, 89, 91]
✓ PASS

==================================================
TEST 4: Edge Cases
==================================================
Empty list:    []
Single element: [42]
Duplicates:    [1, 2, 2, 5, 5]

==================================================
TEST 5: Original Sample Data
==================================================
Sorted Scores: [45, 55, 62, 78, 89]
PS C:\AI Assisted Coding>
```

Merge Sort (Partial Implementation)

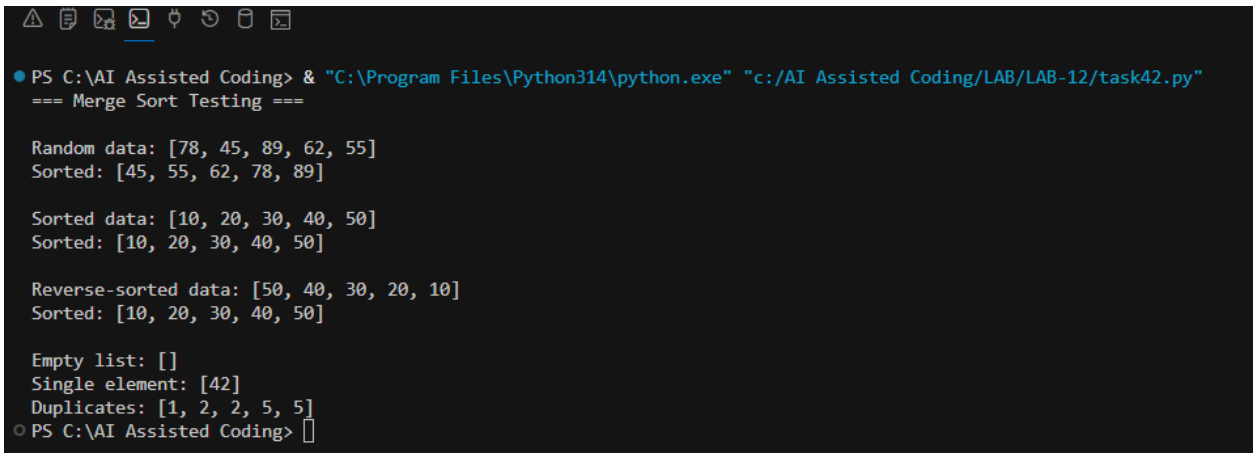
```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    else:
        mid = len(arr) // 2
        left_half = merge_sort(arr[:mid])
        right_half = merge_sort(arr[mid:])
        return merge(left_half, right_half)
def merge(left, right):
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])
    return merged
# Sample Data
scores = [78, 45, 89, 62, 55]
# Testing Merge Sort
sorted_scores = merge_sort(scores)
print("Sorted Scores:", sorted_scores)  # Should return the sorted list of scores
```

Code:

```python
def merge_sort(arr):
    """
    Sort an array using the Merge Sort algorithm (divide-and-conquer).
    Recursion explanation:
    1. Base case: Arrays with ≤1 element are already sorted
    2. Recursive case: Divide array in half, recursively sort each half,
       then merge the sorted halves back together
    Time complexity: O(n log n) | Space complexity: O(n)
    Args:
        arr: List of comparable elements
    Returns:
        Sorted list
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])      # Recurse on left half
    right_half = merge_sort(arr[mid:])     # Recurse on right half
    return merge(left_half, right_half)    # Combine results
def merge(left, right):
    """
    Merge two sorted arrays into a single sorted array.
    Merging explanation:
    - Compare elements from both arrays one at a time
    - Add the smaller element to the result
    - Append remaining elements after one array is exhausted
    Args:
        left: First sorted list
        right: Second sorted list
    Returns:
        Combined sorted list
    """
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])
    return merged
```

```python
# Test cases
print("=== Merge Sort Testing ===\n")
# Test 1: Random data
random_data = [78, 45, 89, 62, 55]
print(f"Random data: {random_data}")
print(f"Sorted: {merge_sort(random_data)}\n")
# Test 2: Already sorted data
sorted_data = [10, 20, 30, 40, 50]
print(f"Sorted data: {sorted_data}")
print(f"Sorted: {merge_sort(sorted_data)}\n")
# Test 3: Reverse-sorted data
reverse_data = [50, 40, 30, 20, 10]
print(f"Reverse-sorted data: {reverse_data}")
print(f"Sorted: {merge_sort(reverse_data)}\n")
# Test 4: Edge cases
print(f"Empty list: {merge_sort([])}")
print(f"Single element: {merge_sort([42])}")
print(f"Duplicates: {merge_sort([5, 2, 5, 1, 2])}")
```

Output:



Time Complexity Comparison:

Case	Quick Sort	Merge Sort
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	$O(n^2)$	O(n log n)
Space Complexity	O(n) (due to new lists)	O(n)

Justification:

Quick Sort performs better on average and is generally faster for random datasets due to lower overhead. However, its worst-case time complexity is $O(n^2)$ when partitions are unbalanced. Merge Sort guarantees O(n log n) performance in all cases and is stable, making it suitable for large datasets and critical applications. Therefore, Quick Sort is preferred for general-purpose sorting, while Merge Sort is chosen when stability and worst-case guarantees are required.

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.

Expected Outcome

• Two versions of the algorithm:

  o Brute-force ($O(n^2)$)

  o Optimized ($O(n)$)

• AI-assisted explanation showing how and why performance

Improved

Code:

```
def naive_duplicate_detection(arr):
    """
    Detects duplicates in the given array using a naive approach with nested loops.
    Returns a list of duplicate elements found in the array.
    """
    duplicates = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j] and arr[i] not in duplicates:
                duplicates.append(arr[i])  # Add to duplicates if not already added
    return duplicates
# Sample Data
scores = [45, 55, 62, 78, 89, 45, 62]
# Testing the naive duplicate detection algorithm
print("Duplicates found:", naive_duplicate_detection(scores))  # Should return [45, 62]
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-12/task5.py"
Duplicates found: [45, 62]
PS C:\AI Assisted Coding\LAB>
```

Code:

```python
def naive_duplicate_detection(arr):
    """
    Detects duplicates using nested loops (Brute-force approach).
    Time Complexity: O(n²) - compares each element with all others
    Space Complexity: O(k) where k is number of duplicates
    """
    duplicates = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j] and arr[i] not in duplicates:
                duplicates.append(arr[i])
    return duplicates
def optimized_duplicate_detection(arr):
    """
    Detects duplicates using a set (Optimized approach).
    Time Complexity: O(n) - single pass through array
    Space Complexity: O(n) for the set
    """
    seen = set()
    duplicates = set()
    for num in arr:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)
# Sample Data
scores = [45, 55, 62, 78, 89, 45, 62]
# Testing both algorithms
print("=== BRUTE-FORCE APPROACH (O(n²)) ===")
print("Duplicates found:", naive_duplicate_detection(scores))
print("\n=== OPTIMIZED APPROACH (O(n)) ===")
print("Duplicates found:", optimized_duplicate_detection(scores))
print("\n=== PERFORMANCE COMPARISON ===")
print("Brute-force: ~49 comparisons for 7 elements")
print("Optimized: ~7 lookups for 7 elements")
print("\nFor 1000 elements:")
print("Brute-force: ~500,000 comparisons")
print("Optimized: ~1000 lookups (500x faster!)")
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-12/task5.py"
=== BRUTE-FORCE APPROACH (O(n²)) ===
Duplicates found: [45, 62]

=== OPTIMIZED APPROACH (O(n)) ===
Duplicates found: [45, 62]

=== PERFORMANCE COMPARISON ===
Brute-force: ~49 comparisons for 7 elements
Optimized: ~7 lookups for 7 elements

For 1000 elements:
Brute-force: ~500,000 comparisons
Optimized: ~1000 lookups (500x faster!)
PS C:\AI Assisted Coding\LAB>
```

Why Performance Improved:

1. Reduced Nested Loop

- Brute force checks every pair → quadratic growth.
- Optimized checks each element once → linear growth.

2. Hashing Instead of Repeated Comparison

Brute force:

- Compares values repeatedly.

Optimized:

- Uses hash table.
- Direct membership checking.
- No redundant comparisons.

3. Better Scalability

As input size increases:

- $O(n^2)$ becomes extremely slow.
- $O(n)$ grows proportionally.

This makes optimized version suitable for:

- Large datasets
- Real-time validation
- Production systems

Approach	Time	Space
Brute Force	$O(n^2)$	O(k)
Optimized	O(n)	O(n)

Justification Statement (For Exam / Notebook)

The optimized algorithm improves performance by reducing time complexity from $O(n^2)$ to $O(n)$ using hashing. Instead of comparing every pair of elements, it uses a set to track previously seen elements and performs constant-time lookups. Although it increases space complexity to $O(n)$, the dramatic reduction in execution time makes it more efficient and scalable for large datasets.